

Performance Report

Cloud Computing

Gabriele Granzotto

Università degli studi di Trieste

March 26, 2026



Outline

- 1 Introduction
- 2 Hardware and Software Setup
- 3 Virtual Machine setup
- 4 Container setup
- 5 Performance Test



- **Goal:** Implementation of parallel clusters: VM vs Docker.
- **Metrics:** Comparative Analysis of CPU, Memory, and Network Usage.
- **Question:** Which technology minimizes virtualization overhead? How much?

CPU:	Intel Core i5-8365U 1.60GHz
RAM:	16GB DDR4
DISK:	256GB SSD NVMe

Table: Hardware Specification

- **Host OS:** Ubuntu 20.04 LTS (64-bit).
- **Guest OS:** Ubuntu 20.04 LTS (64-bit) for both VM and Docker.
- **Programs:** Oracle VirtualBox and Docker CE.
- **Equality:** Limited resources to 2 vCPUs and 4GB RAM for both environments to ensure a fair comparison.

Virtual Machine Setup: Overview

- **Setting OS:** Install the OS, Update, Upgrade and install useful programs and benchmarking tools.
- **Internal Network:** Setting the Internal Network for communication between containers.
- **Shared Filesystem:** Setting a shared folder for data exchange between the host and the VM.
- **Useful Scripts:** Creation of scripts to automate the setup and execution of benchmarks.

Virtual Machine Setup: IP Configuration

Mask:	255.255.255.0
Lower Bound:	192.168.56.2
Upper Bound:	192.168.56.199

Table: CloudBasicNet

Code 1: ssh connection from localhost to master

```
ssh -p 4022 user01@localhost
```

Virtual Machine Setup: Network Configuration

Code 2: /etc/netplan/50-cloud-init.yaml

```
network:
version: 2
ethernets:
enp0s3:
dhcp4: true
enp0s8:
dhcp4: false
addresses: [192.168.56.1/24]
```

- **Netplan:** Configuration of the network interfaces using Netplan.
- **enp0s3:** Interface for NAT connection to the host, configured to obtain an IP address via DHCP.
- **enp0s8:** Interface for the internal network, configured with a static IP address.

Virtual Machine Setup: Hosts File

Code 3: hosts name change

```
127.0.0.1 localhost
127.0.1.1 master
192.168.56.1 master

192.168.56.2 node02
192.168.56.3 node03
192.168.56.4 node04
192.168.56.5 node05
192.168.56.6 node06
192.168.56.7 node07
192.168.56.8 node08
192.168.56.9 node09

# The following lines are desirable for IPv6 capable hosts
::1          ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Virtual Machine Setup: DNS Configuration

Code 4: /etc/dnsmasq.conf

```
interface=enp0s8
bogus-priv
bind-interfaces
cache-size=1000
resolv-file=/etc/resolv.dnsmasq
listen-address=127.0.0.1,192.168.56.1
log-queries
dhcp-range=192.168.56.2,192.168.56.10,12h
```

Code 5: /etc/resolv.conf

```
nameserver 192.168.56.1
search .
```

Virtual Machine Setup: SSH Configuration

Code 6: ssh public key

```
ssh-keygen  
mkdir ~/.ssh  
chmod 700 ~/.ssh  
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys  
chmod 644 ~/.ssh/authorized_keys
```

Code 7: scp command

```
scp -P 4022 /[path]/[to]/[my]/[file] \  
user01@localhost:/[path]/[to]/[send]/
```

Virtual Machine Setup: NFS Configuration

Code 8: /etc/exports

```
...  
/shared/ 192.168.56.0/255.255.255.0(rw,sync,no_root_squash,  
      no_subtree_check)  
...
```

Code 9: /etc/exports

```
sudo systemctl enable nfs-kernel-server  
sudo systemctl restart nfs-kernel-server  
sudo mkdir /shared/data /shared/home  
sudo chmod 777 /shared/data /shared/home
```

Virtual Machine Setup: Useful Scripts

Code 10: upVM.sh

```
#!/bin/bash
echo "Power_On_of_the_VM_cluster..."
VBoxManage startvm "Ubuntu_VM_Master" --type headless
VBoxManage startvm "Ubuntu_VM1" --type headless
VBoxManage startvm "Ubuntu_VM2" --type headless
echo "Power_On_command_send_successfully_to_all_VMs."
```

Code 11: downVM.sh

```
#!/bin/bash
echo "Shutdown_of_the_VM_cluster..."
VBoxManage controlvm "Ubuntu_VM_Master" acpipowerbutton
VBoxManage controlvm "Ubuntu_VM1" acpipowerbutton
VBoxManage controlvm "Ubuntu_VM2" acpipowerbutton
echo "Shutdown_command_send_successfully_to_all_VMs."
```

Container Setup: Overview

- **Container Structure:** Design a multi-container architecture with a master and multiple worker containers.
- **Docker Compose:** Use Docker Compose to orchestrate the multi-container application.
- **Master Node:** Create a Dockerfile for the master node, including necessary software and configurations.
- **Client Node:** Same for the client node, ensuring it can communicate with the master and run benchmarks.
- **SSH problems & Strategy:** Security considerations for SSH key passing between containers and from host to master.

Container Setup: Container Structure

Code 12: Docker project structure

```
Docker-Configuration/  
|  
+--docker-compose.yml  
|  
+--master/  
| |  
| +-- Dockerfile  
+--nodes/  
| |  
| +-- Dockerfile  
+--shared_data/
```

Code 13: docker compose commands

```
docker compose build --no-cache  
docker compose up -d  
docker compose down  
docker exec - it node bash
```

Container Setup: Docker Compose

Code 26: docker-compose.yml

```
services:
  master:
    build:
      context: .
      dockerfile: master/Dockerfile
    container_name: master
    hostname: master
    networks:
      - testnet
  node01:
    build:
      context: .
      dockerfile: nodes/Dockerfile
    container_name: node01
    hostname: node01
    networks:
      - testnet
  node02:
    build:
      context: .
      dockerfile: nodes/Dockerfile
    container_name: node02
    hostname: node02
    networks:
      - testnet
networks:
  testnet:
```

Code 14: docker-compose.yml

```
services:
master:
...
volumes:
- ./shared_data:/data

node01:
...
volumes:
- ./shared_data:/data

node02:
...
volumes:
- ./shared_data:/data
...
volumes:
./shared_data:
```

Container Setup: Docker Compose - Network Config

Code 15: docker-compose.yml

```
services:
  master:
    ...
    cpus: '2.0'
    mem_limit: 2G

  node01:
    ...
    cpus: '2.0'
    mem_limit: 2G

  node02:
    ...
    cpus: '2.0'
    mem_limit: 2G
    ...
  volumes:
    ./shared_data:
```

Container Setup: Master Node

Code 16: master/Dockerfile

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive \
    TZ=Europe/Rome

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
    nano vim curl wget build-essential \
    software-properties-common \
    hpccl mpich stress-ng sysbench iozone3 iperf3 netcat-openbsd \
    openssh-client \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

RUN ssh-keygen -f /root/.ssh/id_rsa -N "" \
    && echo "Host*" > /root/.ssh/config \
    && echo "StrictHostKeyChecking no" >> /root/.ssh/config \
    && echo "UserKnownHostsFile /dev/null" >> /root/.ssh/
    config \
    && chmod 600 /root/.ssh/config

WORKDIR /root/
CMD ["/bin/bash"]
```

Container Setup: Client Node

Code 17: some ssh command

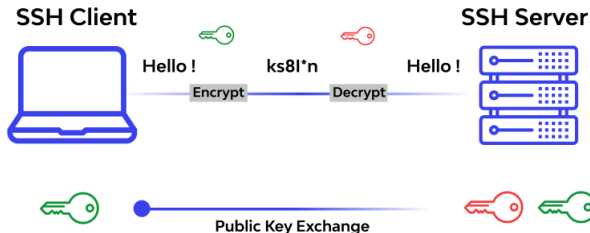
```
RUN mkdir /var/run/sshd \  
&& echo 'root:root' | chpasswd \  
&& sed -i 's/#PermitRootLogin␣prohibit-password/  
    PermitRootLogin␣yes/' /etc/ssh/sshd_config \  
&& sed -i 's/#PasswordAuthentication␣yes/  
    PasswordAuthentication␣yes/' /etc/ssh/sshd_config \  

```

Code 18: node CMD

```
CMD ["/usr/sbin/sshd␣-D"]
```

Container Setup: SSH Key Passing Strategy - Problem



The standard generation of SSH keys works but:

- process cannot be automated during the image build.
- security problems sharing private keys.

Code 19: healthcheck

```
services:
master:
...
healthcheck:
test: ["CMD", "test", "-f", "/data/id_rsa.pub"]
interval: 1s
timeout: 60s
start_period: 3s
...
node01:
...
depends_on:
master:
condition: service_healthy
...
node01:
...
depends_on:
master:
condition: service_healthy
...
```

Container Setup: SSH Key Passing Strategy - Script

Code 20: node CMD (snippet)

```
CMD ["/bin/bash", "-c", "/root/setup.sh␣&&␣/usr/sbin/sshd-D"]
```

Code 21: script.sh

```
#!/bin/bash
mkdir -p /root/.ssh
chmod 700 /root/.ssh
touch /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
cat /data/id_rsa.pub >> /root/.ssh/authorized_keys
```

Code 22: password client (snippet)

```
RUN mkdir /var/run/sshd \
&& echo 'root:\$(openssl rand -base64 12)' | chpasswd \
```

Performance Test: Overview



Figure: free-to-use image credit to Eucalyp

- **CPU:** HPL, FFT, starDGEMM, stress-ng, sysbench.
- **Memory:** MPI random access, starSTREAM triad, stress-ng, sysbench.
- **Disk:** iозone.
- **Network:** iperf3.

Key Takeaways

Important

L'ambiente fragile è necessario quando si usa il codice nelle slide.

Key Objective

Mantenere la coerenza visiva tra il documento scritto e la presentazione orale.